GIÁO TRÌNH THỰC HÀNH · LẬP TRÌNH BACKEND

CHƯƠNG 08

TRUY VẤN NÂNG CAO & PHÂN TRANG XỬ LÝ LỖI TẬP TRUNG

SEQUELIZE · SQL SERVER · EXPRESS 5 · REACT



01 / ĐỊNH HƯỚNG

Mục tiêu, sản phẩm và điều kiện đầu vào

Chương 08 phát triển trực tiếp từ dự án backend-quan-ly-sinh-vien hoàn thành Chương 07. Giữ nguyên SQL Server, JWT, Multer, Vite/React và các REST API cũ; chỉ nâng cấp chức năng đọc danh sách và xử lý lỗi.

Sau hai buổi, sinh viên làm được

- Giải thích page, limit, offset; thực hiện phân trang tại SQL Server bằng Sequelize findAndCountAll.
- Kết hợp search theo hoTen trước khi đếm và chia trang; luôn sắp xếp ổn định theo maSV.
- Trả danh sách là mảng, kèm pagination: page, limit, totalItems, totalPages.
- Rà soát AppError và Global Error Middleware; dùng đúng luồng lỗi của Express 5.
- Kiểm thử có bằng chứng ở Postman/Swagger, SSMS và Network của trình duyệt; review code do AI đề xuất.

Đầu vào phải có

Thành phần	Điều kiện cần xác nhận
Project	Có pnpm build/dev, 5 CRUD SinhVien, OpenAPI YAML, Vite/React trong frontend/.
SQL Server	Sequelize đọc dbo.SinhVien thật; maSV là khóa định danh; có ít nhất 5 bản ghi thử.
Authentication	/api/auth/login, /profile và middleware isAuth/isAdmin còn hoạt động.
Upload	/api/auth/me/avatar và public/uploads vẫn hoạt động từ Chương 07.
Công cụ	VS Code, pnpm, SQL Server/SSMS, Postman, Chrome/Edge DevTools.

PHẠM VI CHÍNH THỨC

Lab 1 (40 phút): Pagination + Search. Lab 2 (30 phút): Global Error Handling. 10 phút: kiểm tra và giải thích. JOIN, Raw Query, Group/Sum và Stored Procedure được đưa vào phần mở rộng có điều kiện.

02 / HỢP ĐỒNG API

Mở rộng chức năng, bảo đảm tính tương thích với hệ thống hiện có

MỤC ĐÍCH CỦA PHẦN THỰC HÀNH

Ở phần này, sinh viên nâng cấp API danh sách sinh viên để tìm kiếm và phân trang ngay tại SQL Server; mỗi lần gọi chỉ trả về những bản ghi cần hiển thị, thay vì tải toàn bộ dữ liệu. Việc mở rộng phải giữ nguyên các chức năng đã làm ở Chương 07: GET danh sách vẫn công khai, POST/PUT/DELETE vẫn chỉ Admin, đồng thời không thay đổi URL, cấu trúc phản hồi hoặc tính năng JWT và Avatar.

Tham số	Quy ước Lab 1	Ví dụ
page	Số nguyên 1, mặc định 1; không hợp lệ 400.	page=2
limit	Số nguyên 1–50, mặc định 10; vượt giới hạn 400.	limit=5
search	Chuỗi tùy chọn theo hoTen; trim; tối đa 80 ký tự.	search=Anh
offset	Backend tự tính: (page 1) × limit.	page=2, limit=5 5
order	Sắp xếp maSV tăng dần trước khi lấy trang.	maSV ASC

Response thành công - hợp đồng thống nhất

```
GET /api/v1/sinh-vien?page=2&limit=2&search=Anh
{
  "success": true,
  "message": "L□y danh sách sinh viên thành công",
  "data": [ /* t□i đa 2 sinh viên phù hợp */ ],
  "pagination": {
    "page": 2, "limit": 2,
    "totalItems": 5, "totalPages": 3
  }
}
```

Các số trên là ví dụ giả định, không phải dữ liệu đã chạy thực tế. Không bao giờ đẩy toàn bộ dữ liệu vào data rồi dùng Array.slice() ở Frontend để giả làm phân trang SQL Server.

> **GIỮ TÍNH TƯƠNG THÍCH**
>
> Khi không truyền query, GET cũ vẫn thành công với data là mảng (kể cả []). Metadata pagination chỉ được thêm ở cấp cao nhất, không bọc data thành object.
>
> Không thay đổi hợp đồng API GET theo mã, POST, PUT, DELETE và /api/auth/me/avatar.

03 / KIẾN TRÚC

Hai luồng xuyên suốt Chương 08

Từng bước phải xác định dữ liệu đi qua tầng nào; Axios không trực tiếp truy vấn SQL Server, còn Error Middleware chỉ dựng response lỗi ở cuối chuỗi.

Luồng A - xem trang 2, có tìm kiếm

```
Vite/React (chọn trang 2, từ khóa Anh)
  -> Axios GET /api/v1/sinh-vien?page=2&limit=10&search=Anh
  -> Express Route -> validate query -> Controller
  -> SinhVien Service -> Sequelize findAndCountAll()
  -> SQL Server: WHERE -> ORDER BY -> OFFSET/FETCH
  -> JSON: data[] + pagination
  -> React hi□n thị Table + Prev/Next + totalItems
```

Luồng B - Request lỗi

```
Client gọi GET ?page=-5
  -> Validate query -> AppError(400, ...)
  -> Global Error Middleware (đặt sau routes)
  -> HTTP 400: {success:false, status:400, message, data:null}

Client gọi GET /sinh-vien/SV999
  -> Service xác định không có -> AppError(404, ...)
  -> Global Error Middleware -> HTTP 404 nh□t quán
```

Khái niệm	Chức năng / Điều cần nhớ
query string	page, limit, search nằm trong req.query; không nằm trong req.params.
Sequelize	where lọc, order ổn định, limit/offset giới hạn tại Database.
AppError	Biểu diễn lỗi có chủ đích, mang mã HTTP như 400 hoặc 404.
Error Middleware	Hàm 4 tham số (err, req, res, next); gửi response thống nhất.
Express 5	Lỗi reject từ async route handler có trả Promise được tự chuyển đến Error Middleware.

ĐIỂM DỪNG CẦN XÁC MINH

Vẽ lại được hai luồng trên và nói rõ khác biệt giữa validation 400, không tìm thấy 404 và lỗi hệ thống 500.

04 / BẢN ĐỒ FILE

Giữ project cũ, chỉ sửa đúng nơi

Tên file chỉ mang tính gợi ý. Nếu repository thực tế đặt tên khác, yêu cầu AI xác định file tương ứng trước khi sửa. KHÔNG tạo backend Chương 08 thứ hai.

```
backend-quan-ly-sinh-vien/
  src/
    app.ts / server.ts             # GIỮ, ki□m tra thứ tự middleware
    routes/sinhVien.routes.ts      # GIỮ, GET dùng Controller cũ
    controllers/sinhVien.controller.ts   # S□A r□t ít
    services/sinhVien.service.ts   # S□A: findAndCountAll
    models/SinhVien.ts             # GIỮ schema (n□u đúng)
    validations/sinhVien.query.ts  # MỚI n□u chưa có Zod query
    utils/AppError.ts              # RÀ SOÁT, không nhân b□n
    middlewares/error.middleware.ts    # RÀ SOÁT/CHU□N HÓA
  docs/
    openapi.yaml                    # CẬP NHẬT page/limit/search
    prompt-log.md                   # B□ SUNG prompt Lab 1/2
    test-results.md                 # B□ SUNG Pxx/Exx
  postman/                          # Export collection sạch
  frontend/src/                    # (Tùy chọn) nút phân trang
  sql/                              # Ch□ script đọc/seed đã review
```

Ba quy tắc không được thay đổi

- pnpm + TypeScript + ESM/NodeNext: import nội bộ đuôi .js khi project hiện tại quy định như vậy.
- SQL Server: không dùng sync({force:true}), sync({alter:true}), DROP, DELETE dữ liệu thật chỉ để test.
- JWT/RBAC và Avatar: không bỏ isAuth/isAdmin, không làm mất route và file upload đã thực hành.

LƯU Ý TÊN ERRORCONTROLLER.JS

Đề cương có ví dụ errorController.js để giải thích Global Error Handler. Vì project đang là TypeScript, ưu tiên error.middleware.ts hiện có; chỉ dùng tên errorController.ts nếu mã nguồn thực tế tổ chức như vậy. Không tạo hai bộ handler cùng làm một việc.

05 / BƯỚC 0

Khảo sát và sao lưu trạng thái Chương 07

Hãy yêu cầu AI đọc hiểu trước, chưa tạo mã. Mọi ca kiểm thử sau này cần có trạng thái ban đầu để so sánh.

Chạy tại thư mục Backend có package.json

```
node -v
pnpm -v
pnpm install
pnpm build
pnpm dev
```

- Gọi GET /api/v1/sinh-vien; kiểm tra success=true, data là mảng; gọi thêm GET /:maSV.
- Thử login và GET /api/auth/profile; Admin DELETE chỉ trên bản ghi test, Student phải bị chặn.
- Chạy truy vấn SSMS chỉ đọc: SELECT COUNT(*) AS total FROM dbo.SinhVien;
- Đảm bảo có 5 mã sinh viên test sắp theo maSV, có ít nhất một tên chứa “Anh”. Không giả định SV001–SV005 luôn tồn tại.
- Chụp trạng thái Swagger, tree project và commit/backup phiên bản chạy tốt.

PROMPT 0 · Chỉ khảo sát - chưa viết code

C · CONTEXT	Tôi tiếp tục đúng backend-quan-ly-sinh-vien sau Chương 07: Express 5, TypeScript, Sequelize 6, SQL Server, Zod, JWT, Vite/React.
T · TASK	Đọc package.json, tsconfig, GET routes/controller/service, SinhVien Model, AppError, error middleware, docs/openapi.yaml; vẽ luồng GET.
C · CONSTRAINT	Chưa sửa bất cứ file nào; không đọc .env; không giả định có Products, Orders hoặc lớp học nếu không tìm thấy.
O · OUTPUT	Bảng file/đúng vai trò/cần sửa hay giữ; xác định thiếu gì; checklist API nền và truy vấn SSMS cần test.

ĐIỂM DỪNG BẮT BUỘC

Chỉ tiếp tục nếu pnpm build và các API nền chạy tốt. Nếu chưa đạt, ghi lỗi thật vào docs/test-results.md và sửa lỗi gốc trước.

06 / BƯỚC 1 · LAB 1

Hiểu page, limit, offset và SQL Server

Mục đích của phân trang là giới hạn số bản ghi truyền về ở Database. React không được tải tất cả rồi tự cắt.

Công thức cần tự giải thích

```
offset = (page - 1) * limit
page = 1, limit = 2  => offset = 0
page = 2, limit = 2  => offset = 2
page = 3, limit = 2  => offset = 4
```

Ví dụ một bảng có 5 sinh viên được sắp theo mã: trang 1 lấy 2, trang 2 lấy 2, trang 3 lấy 1. totalPages = Math.ceil(5 / 2) = 3.

Xem SQL thuần tương đương (chạy SELECT trong SSMS)

```
SELECT maSV, hoTen, email, maLop
FROM dbo.SinhVien
ORDER BY maSV ASC
OFFSET 2 ROWS
FETCH NEXT 2 ROWS ONLY;
```

Kỹ thuật	Ý nghĩa kiểm chứng
ORDER BY maSV	Thứ tự ổn định giữa các trang; phải có khi dùng OFFSET/FETCH.
OFFSET 2	Bỏ qua đúng hai bản ghi đầu.
FETCH NEXT 2	Chỉ đọc tối đa hai bản ghi của trang hiện tại.
COUNT(*)	Tổng bản ghi phù hợp, phục vụ tính tổng số trang.

BÀI TÍNH NHANH TRÊN LỚP

Nếu page=5 và limit=20 thì offset bằng bao nhiêu? Ghi phép tính vào báo cáo.

Nếu totalItems=0, quy ước bài này trả data=[] và totalPages=0; page vẫn là giá trị yêu cầu.

07 / BƯỚC 2 · LAB 1

Validate query string trước khi truy vấn

Các giá trị lấy từ req.query có thể là chuỗi, mảng hoặc không tồn tại. Không đưa thẳng vào limit/offset khi chưa kiểm tra và chuẩn hóa.

Đầu vào	Kết quả	Lý do
?page=1&limit=10	200	Hợp lệ.
?page=-1	400	page phải là số nguyên dương.
?page=1.5	400	Không cho số thập phân.
?limit=abc	400	Không thể là số nguyên.
?limit=51	400	Vượt giới hạn 50 của lab.
?page=0	400	Trang bắt đầu từ 1.
?search=%20Anh%20	200	Được trim thành Anh.
?search=... (81 ký tự)	400	Từ khóa quá dài.

Chiến lược thực hiện

- Tận dụng Zod ở Chương 05; nếu đã có validateQuery, tái sử dụng. Chuyển chuỗi số hợp lệ thành number sau khi kiểm tra chặt.
- Giữ 1 page, 1 limit 50, search tối đa 80; mặc định page=1, limit=10, search rỗng.
- Phát sinh AppError(400, ...) hoặc cơ chế 400 đã tồn tại; không trả 500 vì dữ liệu người dùng sai.
- Tránh ép kiểu kiểu Number(" ") = 0 hoặc parseInt("2abc") = 2 mà không báo lỗi.

PROMPT 1 · Zod / validate query

C · CONTEXT	GET SinhVien cũ nhận req.query; project có Zod và AppError từ những chương trước.
T · TASK	Tạo hoặc tái sử dụng query validation cho page, limit, search với mặc định 1/10/"", giới hạn limit 50 và search 80.
C · CONSTRAINT	Sai trả 400 thống nhất; không sửa middleware JWT/Avatar; không tự đổi TypeScript, ESM và cấu trúc response.
O · OUTPUT	Code theo file thật, mô tả cách gắn tại route/controller và ca thử hợp lệ/sai.

08 / BƯỚC 3 · LAB 1

Phân trang thật bằng Sequelize findAndCountAll

Nâng cấp Service hiện có, giữ Model dbo.SinhVien và định danh maSV. Đây là phần bắt buộc, không thay bằng thao tác .slice() trên mảng JavaScript.

Logic tham chiếu ngắn gọn - đọc hiểu, không chép nguyên khi khác tên Model

```
const offset = (page - 1) * limit;
const { rows, count } = await SinhVien.findAndCountAll({
  where,                    // where theo search (n□u có)
  limit,
  offset,
  order: [["maSV", "ASC"]]
});
return {
  data: rows,
  pagination: {
    page, limit,
    totalItems: count,
    totalPages: Math.ceil(count / limit)
  }
};
```

- rows: mảng sinh viên của trang được yêu cầu, không phải toàn bộ bảng.
- count: tổng số sinh viên thỏa điều kiện where trước khi phân trang.
- Trang vượt quá tổng số trang: data=[] (không báo 500), metadata vẫn phản ánh tổng số dữ liệu.
- Bật logging truy vấn Sequelize (nếu cần) để chứng minh SQL Server xử lý giới hạn, rồi tắt log nhạy cảm khi nộp.

PROMPT 2 · Service Sequelize

C · CONTEXT	Project đang dùng SinhVien Model và Sequelize 6 kết nối SQL Server thật.
T · TASK	Nâng cấp hàm lấy danh sách của Service bằng findAndCountAll, where từ search, order maSV ASC, limit/offset.
C · CONSTRAINT	Giữ maSV/hoTen/email/maLop, không thêm id/timestamps/association giả; không gọi findAll toàn bộ rồi slice.
O · OUTPUT	Đoạn code đúng file, giải thích rows/count/totalPages, cách xác minh SQL bằng Postman + SSMS.

ĐIỂM DỪNG

Dữ liệu trang 1 và trang 2 có thứ tự đúng, không lặp khi không có cập nhật dữ liệu xen giữa; count phải đến từ Database.

09 / BƯỚC 4 · LAB 1

Controller, response và Swagger YAML

Controller đọc query đã validate, gọi Service và trả JSON tương thích. Đặc tả API vẫn nằm trong docs/openapi.yaml đã dùng từ Chương 04, không chuyển sang JSDoc một cách tùy tiện.

Cách tổ chức phần HTTP

- Giữ route GET /api/v1/sinh-vien cũ và chế độ public; thay đổi tối thiểu controller/service.
- Giữ response GET thành công có success, message và data là mảng; thêm pagination ở cấp cao nhất.
- Bổ sung query parameters trong YAML: page integer 1, limit integer 1–50, search string 80.
- Định nghĩa response 200 và 400, ví dụ metadata, trường data array; giữ security của POST/PUT/DELETE và Bearer cho Avatar.
- Nếu hợp đồng GET cũ không có pagination thì client cũ vẫn có thể đọc response.data.data như trước.

```
GET /api/v1/sinh-vien?page=1&limit=10
{
  "success": true,
  "message": "L□y danh sách sinh viên thành công",
  "data": [],
  "pagination": {
    "page": 1, "limit": 10,
    "totalItems": 0, "totalPages": 0
  }
}
```

Ví dụ này minh họa riêng trường hợp không có bản ghi nào; data=[], không phải null.

PROMPT 3 · Controller + OpenAPI

C · CONTEXT	GET /api/v1/sinh-vien hiện có data[]; tài liệu API là docs/openapi.yaml.
T · TASK	Gắn query đã validate vào Controller, trả data[] + pagination, cập nhật YAML chuẩn OpenAPI 3.
C · CONSTRAINT	Không đổi URL/method/status các API cũ; không khóa GET bằng JWT; không chuyển sang swagger-jsdoc.
O · OUTPUT	Diff theo file, ví dụ request/response 200/400, cách mở /api-docs và test.

10 / BƯỚC 5 · LAB 1

Tìm kiếm theo tên rồi mới phân trang

Nếu lọc chỉ trong các bản ghi của trang hiện tại, hệ thống sẽ bỏ sót kết quả ở trang khác. WHERE phải được áp dụng trước COUNT và LIMIT/OFFSET.

```
GET /api/v1/sinh-vien?search=Anh&page=2&limit=2

SQL Server: lọc hoTen phù hợp -> đ□m t□ng k□t qu□
           -> ORDER BY maSV -> l□y trang 2

Sequelize: where + findAndCountAll + limit/offset
```

Yêu cầu kỹ thuật

- Chỉ lọc theo hoTen như slide; không tự thêm lọc theo email hoặc lớp trong bài chính.
- Dùng Op.like với chuỗi do code quản lý, không tự ghép Raw SQL bằng nội dung nhập vào.
- Xác nhận cách so khớp chữ hoa/thường phụ thuộc collation hiện có của SQL Server; không khẳng định luôn giống nhau.
- Nếu hỗ trợ tìm chuỗi chứa %, _ hoặc [, xử lý ký tự đặc biệt theo SQL Server hoặc tài liệu rõ quy ước; không coi chúng mặc nhiên là chữ thường.
- Khi Frontend đổi search, quay về trang 1 để không bị "rỗng" do đang đứng trang quá xa.

PROMPT 4 · Tìm kiếm kết hợp phân trang

C · CONTEXT	SQL Server đã có bảng SinhVien, cột hoTen; GET danh sách đã trả rows/count.
T · TASK	Thêm search tùy chọn bằng Sequelize where + Op.like, đếm theo tập kết quả phù hợp, rồi order/paginate.
C · CONSTRAINT	Không lọc trên rows đã chia trang; không dùng raw SQL nội suy text; giữ validation và API cũ.
O · OUTPUT	Diff Service, ít nhất 4 ca test: có kết quả, không có, search có khoảng trắng, search + page 2.

TỰ GIẢI THÍCH

Một sinh viên "Lan Anh" đang ở vị trí thứ 30 theo maSV. Vì sao cách lấy 10 dòng đầu rồi mới filter sẽ không tìm thấy bạn ấy?

11 / BƯỚC 6 · TÙY CHỌN

Gắn phân trang vào Vite + React + Axios

Frontend đã được tạo ở Chương 07, vì vậy có thể bổ sung phần này nếu Lab 1 hoàn thành sớm. Slide Chương 08 xác định đây là phần dành cho nhóm xong trước, không bắt buộc trong thời lượng 40 phút.

UI tối thiểu có thể nâng cấp

- Ô tìm kiếm theo hoTen, nút Trước / Sau, nhãn "Trang X/Y" và tổng số bản ghi.
- Khi bấm trang 2, Axios gửi lại GET với page=2&limit;=...; không dùng .slice() cắt toàn bộ dữ liệu cũ.
- Đọc response.data.data thành mảng và response.data.pagination thành metadata.
- Hiển thị đầy đủ Loading / Empty / Error; vô hiệu hóa nút Trước ở trang 1 và Sau ở trang cuối.
- Giữ trạng thái Login/Admin DELETE/Avatar của Chương 07; không bỏ auth hoặc mở CORS tùy tiện.

```
const response = await api.get("/api/v1/sinh-vien", {
  params: { page, limit: 10, search }
});
setStudents(response.data.data);
setPagination(response.data.pagination);
```

PROMPT 5 · React + Axios - mở rộng

C · CONTEXT	frontend/ Vite React từ Chương 07 có component SinhVienTable và Axios client.
T · TASK	Bổ sung search, nút đổi trang, metadata và trạng thái loading/error theo API mới.
C · CONSTRAINT	Không thay backend khác; không dùng Array.slice() để phân trang dữ liệu tải hết; giữ Login, Delete, Avatar.
O · OUTPUT	Code component đúng file, state và sự kiện, các thao tác DevTools Network cần chụp.

ĐIỂM DỪNG

Đổi trang trên UI làm phát sinh một request HTTP có page mới; Network cho thấy data trả về thực sự đã bị giới hạn.

12 / KIỂM THỬ LAB 1

Ma trận Pagination và Search - phần A

Dùng Postman hoặc Swagger; lưu mã ca P01–P09 và ảnh/JSON thực tế. Chuẩn bị ít nhất 5 bản ghi test có maSV tăng dần; trong kiểm thử phân trang, không đồng thời thêm/xóa bản ghi.

Mã	Request / Tình huống	Expected	Cách xác minh
P01	GET /api/v1/sinh-vien	200	data là mảng; pagination mặc định page=1, limit=10.
P02	GET ?page=1&limit=2	200	Tối đa 2 bản ghi đầu theo maSV.
P03	GET ?page=2&limit=2	200	Tối đa 2 bản ghi tiếp; không trùng P02.
P04	GET ?page=3&limit=2	200	Trang cuối có thể ít hơn 2.
P05	GET ?page=999&limit=2	200	data=[]; totalItems vẫn chính xác.
P06	GET ?page=-1	400	Thông báo page không hợp lệ.
P07	GET ?page=0	400	page phải 1.
P08	GET ?page=1.5	400	Không chấp nhận page thập phân.
P09	GET ?limit=abc	400	Không trả 500, không thực hiện query sai.

Mỗi ca kiểm thử cần ghi

```
ID | Method + URL | Input | Expected HTTP + JSON
   | Actual HTTP + JSON | PASS / FAIL | Minh chứng | Cách s□a
```

KHÔNG ĐƯỢC ĐÁNH PASS CHỈ VÌ THẤY HTTP 200

Đối chiếu maSV các trang và kiểm tra totalItems/totalPages với SELECT COUNT(*) trong SSMS. Thứ tự phải được xác định rõ.

13 / KIỂM THỬ LAB 1

Ma trận Pagination và Search - phần B

Các ca sau kiểm tra chặt query, điều kiện lọc và chứng minh dữ liệu thật ở SQL Server.

Mã	Tình huống / Request	Expected	Bằng chứng cần có
P10	GET ?limit=51	400	Chặn limit vượt 50.
P11	GET ?search=Anh	200	Chỉ tên phù hợp theo collation đã kiểm chứng.
P12	GET ?search=Anh&page=2&limit=2	200	totalItems là tổng KẾT QUẢ LỌC, không phải tổng bảng.
P13	GET ?search=khongco	200	data=[]; totalItems=0; totalPages=0.
P14	GET ?search=%20Anh%20	200	Trim thành Anh.
P15	GET ?search=[81 ký tự]	400	Chặn từ khóa quá dài.
P16	GET ?page=2&limit=2 rồi xem SQL log	200	SQL có ORDER BY + OFFSET/FETCH hoặc SQL tương đương.
P17	DELETE/POST/PUT/GET theo mã cũ	Như cũ	JWT/Admin/validation và CRUD không bị hồi quy.

Cách kiểm chứng Database

```
-- Thực hiện với cùng trạng thái dữ liệu dùng đ test
SELECT COUNT(*) AS totalItems FROM dbo.SinhVien;
SELECT maSV, hoTen FROM dbo.SinhVien ORDER BY maSV ASC;
-- N u mu n đ m search, dùng query có WHERE tương ứng
-- với đi u kiện thực t  c a project / collation.
```

Lưu ý: Khi đang có thao tác ghi đồng thời, các trang offset có thể thay đổi. Lab cơ bản kiểm thử trên tập dữ liệu ổn định.

ĐIỂM DỪNG SANG LAB 2

Phải PASS P02, P03, P06, P09, P11, P12 và P17 ở mức chức năng cốt lõi. Nếu sai, sửa Lab 1 trước khi refactor Error Handling.

14 / BƯỚC 7 · LAB 2

Khảo sát AppError và Middleware đã có

Trong Chương 07, dự án có thể đã xử lý lỗi từ JWT, Zod và Multer. Không xóa bộ cũ chỉ vì muốn tạo file errorController.js mới.

Mở và đọc ba nơi trước khi yêu cầu AI sửa

File / thành phần	Điểm phải đọc
src/utils/AppError.ts	Constructor nhận statusCode/message theo thứ tự nào; có operational flag hay không?
src/middlewares/error.middleware.ts	Là middleware 4 tham số? Có 400, 404, 500? Có che lỗi hệ thống?
src/controllers/sinhVien.controller.ts	Có try-catch lặp, next(err), throw AppError hay response lỗi thủ công?
src/app.ts hoặc server.ts	Thứ tự use router, 404 fallback và errorHandler như thế nào?
Auth + Multer + Zod	Các lỗi 401/403/413/409 đang hoạt động có bị handler mới thay đổi?

Ranh giới trách nhiệm

```
AppError              -> bi□u di□n một l□i dự ki□n
Controller / Service   -> throw AppError khi c□n
Express 5             -> chuy□n Promise rejection hợp lệ
Global Error Handler  -> quy□t định HTTP code + JSON cu□i cùng
```

PROMPT 6 · Audit Global Error Handling

C · CONTEXT	Tôi tiếp tục Express 5 TypeScript trong project Chương 07; đã có AppError và error.middleware.ts (hoặc file tương đương).
T · TASK	Đọc nguyên file hiện tại và kiểm tra luồng 400, 404, 500 cùng các lỗi JWT, Zod, Multer.
C · CONSTRAINT	Chỉ đề xuất sửa nhỏ nhất; chưa viết lại file khi không cần; không chuyển Express 5 thành Express 4.
O · OUTPUT	Bảng lỗi hiện tại/điểm cần sửa/rủi ro; sơ đồ luồng err -> errorHandler, ca test bắt buộc.

15 / BƯỚC 8 · LAB 2

Chuẩn hóa response lỗi mà không lộ bí mật

Hợp đồng cũ của Chương 04–07 dùng success/message/data. Bài này giữ lại ba trường đó. Trường status dạng mã HTTP là phần bổ sung phục vụ yêu cầu Chương 08.

Mẫu error response có chủ đích

```
HTTP/1.1 400 Bad Request
{
  "success": false,
  "status": 400,
  "message": "Tham s page không hợp lệ",
  "data": null
}
```

Quy định khi gặp lỗi 500

- Dùng HTTP 500 và message công khai "Lỗi hệ thống, vui lòng thử lại sau"; không trả SQL text, stack trace, .env, mật khẩu, connection string hoặc đường dẫn máy chủ trong production.
- Ghi chi tiết lỗi vào log phía server theo quy định; kiểm tra bảo vệ dữ liệu cá nhân trong log.
- Nếu res.headersSent, chuyển next(err) thay vì gửi JSON lần hai.
- Giữ hành vi status 401/403 (JWT), 409 (trùng maSV), 413 (upload >2MB) như bài trước.

PHÂN BIỆT SLIDE VÀ YÊU CẦU MỞ RỘNG VỀ STACK

Slide Chương 8: không đưa stack trace tới Client. Đây là mức mặc định và an toàn hơn.

Yêu cầu bổ sung trong đề bài: chỉ cho phép trường stack khi NODE_ENV=development, môi trường cục bộ không công khai; tuyệt đối không có ở production. Trường này là mở rộng tùy chọn, không được làm lộ bí mật hay đưa vào bản deploy.

PROMPT 7 · Chuẩn hóa error middleware

C · CONTEXT	API đang có {success,message,data}; đã tồn tại AppError và Error Handler; dùng Express 5.
T · TASK	Chuẩn hóa JSON lỗi 400/404/500, có status code; 500 phải che message gốc và có server logging.
C · CONSTRAINT	Không lộ stack production; chỉ thêm stack development nếu thỏa môi trường an toàn; giữ JWT/Multer errors và không trả hai response.
O · OUTPUT	Diff file hiện có, ví dụ 400/404/500 và giải thích khác nhau giữa dev và production.

16 / BƯỚC 9 · LAB 2

Tận dụng async error flow của Express 5

Express 5 tự chuyển Promise bị reject từ async route handler trả về Promise sang Error Middleware. Không bọc thêm try-catch/asyncHandler vào mọi Controller một cách máy móc.

Minh họa đúng trọng tâm

```
// Ví dụ: chỉ áp dụng khi Controller là async handler
const listSinhVien = async (req, res) => {
  const result = await service.list(req.query);
  res.json({ success: true, message: "OK", ...result });
};

// Route có handler trả Promise -> Express 5
router.get("/", validateQuery, listSinhVien);

// Cuối ứng dụng, SAU các routes và 404 handler:
app.use(errorHandler);
```

- Phát sinh lỗi nghiệp vụ: throw new AppError(...) theo constructor đang có; không tự chế constructor mới và làm hỏng nơi cũ.
- Nếu tự catch để thêm ngữ cảnh hoặc phục hồi, phải next(err) hoặc ném lại, không nuốt lỗi.
- Không trả JSON lỗi trong Service theo req/res; giữ tách tầng Route Controller Service.
- Đặt 404 fallback cho endpoint không tồn tại theo kiến trúc hiện hữu; giữ GET /:maSV không tìm thấy trả 404.
- Khi trình bày, phân biệt lỗi 400 do validate, 404 không có dữ liệu, 500 do hệ thống.

PROMPT 8 · Refactor ít nhất + kiểm thử

C · CONTEXT	Đã kiểm tra AppError, controller và errorHandler hiện tại; dự án chạy Express 5.
T · TASK	Loại bỏ code trả lỗi lặp nếu thực sự cần; bảo đảm lỗi được chuyển tới một handler sau routes.
C · CONSTRAINT	Không tạo nhiều global handlers; không cần asyncHandler cho mọi Promise; giữ các status cũ, không che giấu lỗi.
O · OUTPUT	Liệt kê những dòng thay đổi và các ca kiểm thử E01–E10 cần thực hiện.

17 / KIỂM THỬ LAB 2

Ma trận lỗi 400, 404, 500 và hồi quy bảo mật

Bằng chứng quan trọng nhất không phải AI nói “đã làm”, mà là HTTP status và nội dung JSON đúng khi cố tình tạo lỗi.

Mã	Tình huống	HTTP	Kết quả kiểm chứng
E01	GET ?page=-5	400	success=false; message rõ; data=null.
E02	GET ?limit=abc	400	Bị validate trước Database.
E03	GET /api/v1/sinh-vien/SV999	404	Không tìm thấy; thông báo nhất quán.
E04	GET /duong-dan-khong-co	404	Fallback 404 còn hoạt động.
E05	Lỗi 500 mô phỏng trong dev	500	Không lộ SQL/secret; production che message.
E06	NODE_ENV=production, test 500	500	Không có stack; message an toàn.
E07	NODE_ENV=development, test 500	500	Nếu bật mở rộng stack, chỉ tồn tại ở dev cục bộ.
E08	GET /api/auth/profile thiếu token	401	Auth cũ vẫn chặn.
E09	Student ép DELETE bản ghi test	403	RBAC cũ vẫn chặn, DB không đổi.
E10	Upload >2MB / sai file như Ch07	413/400	Multer + error JSON theo quy định cũ.

CÁCH KIỂM THỬ LỖI 500 AN TOÀN

Không cần tắt SQL Server hoặc phá dữ liệu. Chỉ sử dụng test route tạm chạy cục bộ trong development để throw Error mô phỏng; ghi rõ route này vào test log.

XÓA route test 500 sau khi kiểm thử, trước khi nộp/deploy; nếu kiểm tra production, chạy bản build riêng trong môi trường test an toàn.

18 / HỒI QUY VÀ SỬA LỖI

Kiểm tra lại Chương 07 sau khi nâng cấp

Một bài đạt Chương 08 nhưng làm hỏng Login, Upload hoặc CRUD của Chương 07 vẫn chưa hoàn thành.

Nhóm	Ca bắt buộc hồi quy	Điều cần thấy
Danh sách	GET /api/v1/sinh-vien	200, data là mảng; có pagination.
CRUD	GET/:maSV; POST; PUT; DELETE	Các status 200/201/400/404/409 giữ nguyên.
JWT / RBAC	Profile; Student DELETE; Admin DELETE	401 / 403 / 200 theo bài cũ.
Multer	Upload Avatar đúng/sai kích thước	200 / 400 / 413; SQL lưu avatarUrl.
Frontend	Login, Table, Admin Delete, Avatar	Không crash vì response thêm pagination.
Swagger	GET query + routes cũ	YAML mô tả đúng; đường dẫn còn đầy đủ.

Các lỗi AI thường đề xuất sai

- Thay đổi GET danh sách từ data[] thành data{rows,...} làm frontend Chương 07 vỡ.
- Dùng findAll() toàn bảng và slice phía JavaScript, tạo phân trang giả.
- Không ORDER BY nên các trang thiếu ổn định; đếm count trên dữ liệu đã bị giới hạn.
- Gắn sai Global Error Handler trước router, tạo thêm Express app hoặc trùng handler.
- Cho Client thấy stack trace/SQL message trong production, bỏ kiểm tra JWT/Multer.
- Làm việc trên dist/ hoặc thay đổi pnpm/ESM/TypeScript không cần thiết.

PROMPT 9 · Chẩn đoán lỗi theo bằng chứng

C · CONTEXT	Đang ở bước [ghi bước] của Chương 08 và project đang có lỗi [dán đầy đủ].
T · TASK	Giải thích nguyên nhân từ terminal, method/URL, HTTP response và file thật; đề xuất bản sửa ít nhất.
C · CONSTRAINT	Không phỏng đoán schema, không vô hiệu hóa JWT/validation, không sửa project đang chạy tốt ở chỗ khác.
O · OUTPUT	Diff file và checklist build + ca test lỗi cũ + 1 ca regression liên quan.

19 / MỞ RỘNG BUỔI 10

JOIN · Raw Query · Thống kê · Stored Procedure

Đề cương có các lựa chọn này nhưng hai Lab trên slide bắt buộc chỉ yêu cầu Pagination + Search và Error Handling. Chỉ làm phần mở rộng khi đã kiểm tra bảng/cột thực tế, không tự tạo mô hình bán hàng khác.

A. Raw Query - báo cáo tổng số sinh viên theo lớp

```
SELECT maLop, COUNT(*) AS totalStudents
FROM dbo.SinhVien
GROUP BY maLop
ORDER BY maLop ASC;
```

Dùng sequelize.query() để đọc kết quả, đối chiếu trực tiếp SSMS. Nếu muốn làm “doanh thu theo tháng” như prompt đề cương, chỉ thực hiện khi Database thực sự có bảng Orders/OrderItems, ngày đặt và số tiền; không tự giả định schema.

B. Sequelize Method - group/count theo maLop

Dùng findAll({ attributes: ["maLop", [fn("COUNT", col("maSV")), "totalStudents"]], group: ["maLop"] }) để thống kê. Nếu project có bảng đơn hàng liên kết khách hàng thật, có thể nâng lên đếm đơn theo khách hàng; chú ý đây không phải cùng bài toán với tổng doanh thu.

C. Stored Procedure - thủ tục chỉ đọc

Hỏi AI cách tạo thủ tục chỉ đọc dbo.sp_ThongKeSinhVienTheoLop và cách gọi từ Sequelize kết nối SQL Server. Script phải qua review, có lệnh kiểm chứng trong SSMS, không DROP dữ liệu hoặc thay đổi bảng đang dùng.

D. JOIN và 5 bản ghi mới nhất

Chỉ JOIN Lop qua maLop nếu bảng Lop và Sequelize association thực sự tồn tại. Chỉ dùng order createdAt DESC nếu Model có createdAt hoặc cột thời gian tương đương.

MINH CHỨNG CHO PHẦN MỞ RỘNG

Nộp câu truy vấn/Sequelize method hoặc Stored Procedure, kết quả SSMS và Postman (nếu tạo endpoint), giải thích lý do chọn cách đó. Bài mở rộng không thay thế hai Lab chính.

20 / KỸ NĂNG AI-DRIVEN

Quy trình CTCO và code review bắt buộc

Sản phẩm học tập là code đã được hiểu và kiểm thử, không phải chỉ có đoạn trả lời từ chatbot.

Bước	Câu hỏi sinh viên phải tự trả lời
1. Phân tích	Đang sửa endpoint và file nào? Điều gì phải giữ nguyên?
2. Prompt CTCO	Context, Task, Constraint, Output đã đủ và đúng project chưa?
3. Soát diff	AI thay đổi bao nhiêu file? Có tự tạo Products/Orders không?
4. Build/Test	pnpm build có PASS? SQL Server có dữ liệu thật? Status hợp đồng đúng?
5. Giải thích	Vì sao cần ORDER BY, count, AppError, error middleware?

Hai câu hỏi phản biện AI cần lưu trong prompt-log.md

- "AI vừa dùng findAll() rồi .slice() để chia trang. Hãy chỉ ra tại sao không đạt tiêu chí và viết lại truy vấn phân trang ở Database."
- "AI vừa trả err.stack và err.message cho mọi lỗi 500. Hãy chỉ ra rủi ro bảo mật, sửa để production không tiết lộ nội bộ và kiểm thử lại."

Mẫu nhật ký đề xuất

```
Ngày / Nhóm / Prompt s□: _________
V□n đ□ thực t□ c□n gi□i quy□t: _____________________
AI đ□ xu□t s□a file: _____________________________
Những đi□m đã giữ / đã từ ch□i: ___________________
Build: PASS / FAIL
Case ki□m th□ liên quan: P___, E___ ; k□t qu□: ____
Đi□u tự gi□i thích được: _____________________________
```

KHÔNG CHÉP BÍ MẬT VÀO PROMPT

Ẩn .env, JWT thật, mật khẩu SQL Server, file ảnh cá nhân và thông tin người dùng khi gửi file cho AI hoặc xuất Postman Collection.

21 / NỘP BÀI

Hồ sơ bắt buộc và thang chấm 10 điểm

Ưu tiên API chạy với SQL Server thật, bằng chứng đầy đủ và review được code AI. Nộp một repository duy nhất tiếp nối từ Chương 07.

Sinh viên phải nộp

- Mã nguồn project đã nâng cấp (giữ frontend/); không nộp .env, node_modules, dist, token thật, ảnh riêng tư.
- docs/openapi.yaml mô tả GET page/limit/search và lỗi 400, đồng thời giữ các endpoint JWT/Avatar.
- Postman Collection và docs/test-results.md có P01–P17, E01–E10 cùng Actual và PASS/FAIL trung thực.
- docs/prompt-log.md ghi ít nhất 5 prompt đã dùng, trong đó tối thiểu 2 lần phát hiện/chỉnh sửa đề xuất AI.
- Ảnh bằng chứng: SSMS + 2 trang dữ liệu khác nhau; Postman search; lỗi 400/404/500; Swagger; nếu có Frontend, chụp Network khi đổi trang.
- Video demo 2–3 phút hoặc chuỗi ảnh có lời giải thích 4 chức năng: pagination, search, xử lý lỗi, không hồi quy JWT/Avatar.

Rubric 10 điểm

Tiêu chí	Điểm	Minh chứng tối thiểu
Pagination SQL Server thật	2.5	P02/P03/P16, order ổn định, count đúng.
Search + query validation	2.0	P06/P09/P11/P12/P13; filter trước phân trang.
Global Error Handling	2.0	E01/E03/E05/E06; JSON nhất quán, không lộ stack production.
Hồi quy Chương 07	1.5	E08–E10 và CRUD/JWT/Upload hoạt động.
Swagger, bằng chứng, prompt-log, bảo vệ	2.0	YAML, test-results, prompt-log và giải thích được code.

KHÔNG ĐẠT TRỌNG TÂM

Không đạt yêu cầu nếu chỉ chia trang bằng Frontend, dùng dữ liệu mock thay SQL Server, thay API SinhVien thành Products, tắt JWT để test, hoặc gửi stack/SQL secret cho Client production.

22 / TỰ KIỂM TRA

12 câu hỏi bảo vệ bài thực hành

Sinh viên trả lời bằng lời của mình và chỉ ra đúng file/dòng code, không đọc lại câu trả lời từ AI.

- 01. page, limit và offset khác nhau thế nào? page=5 và limit=20 thì offset là bao nhiêu?
- 02. Vì sao phải ORDER BY maSV trước khi dùng OFFSET/FETCH trong SQL Server?
- 03. findAll() khác findAndCountAll() ra sao? rows và count dùng để làm gì?
- 04. Vì sao totalPages = Math.ceil(totalItems / limit)? Khi không có dữ liệu thì trả gì?
- 05. Vì sao lọc search theo hoTen phải xảy ra trước khi tính count và lấy trang?
- 06. Nếu client gửi page="abc" hoặc limit=-2, lỗi phù hợp là 400 hay 500? Vì sao?
- 07. Vì sao không nên trả data={rows,pagination} thay cho data[] của chương trước?
- 08. Axios gửi SQL tới Database hay gửi HTTP tới Backend? Chứng minh bằng Network.
- 09. AppError và Global Error Middleware khác nhau về trách nhiệm?
- 10. Express 5 chuyển lỗi từ async handler tới middleware như thế nào?
- 11. Vì sao không được hiển thị stack trace và connection string ở production?
- 12. Nêu ba bằng chứng để chứng minh nâng cấp Chương 08 không làm hỏng Chương 07.

Bài tập mở rộng (không thay thế bài chính)

- So sánh COUNT theo maLop bằng Raw Query và Sequelize group, đối chiếu SSMS.
- Nếu có bảng Lop và association: JOIN để lấy tenLop; giải thích include và điều kiện khóa ngoại.
- Nếu có cột ngày tạo: API 5 sinh viên mới nhất; nếu không có, giải thích vì sao không thể đoán createdAt.
- Viết Stored Procedure chỉ đọc thống kê theo lớp, đánh giá lợi ích/hạn chế so với Sequelize.

> **QUY TRÌNH AI-DRIVEN CẦN TỰ CHỊU TRÁCH NHIỆM**
>
> HIỂU YÊU CẦU VIẾT PROMPT CTCO ĐỌC & REVIEW CODE PNPM BUILD KIỂM THỬ SQL SERVER THẬT GIẢI THÍCH KẾT QUẢ